A Log-Normal Model of Server CPU Utilization Under Transaction Load: Fit, Scaling, and SLA-Aware Sizing

Alexander Apartsin
apartsin@gmail.com

Draft, August 2026

Abstract

Sizing servers for a transaction-processing service is a costly trade-off: over-provisioning wastes infrastructure budget, under-provisioning drops transactions and triggers SLA penalties. Sound sizing rests on one quantity: given a transaction rate, the probability that CPU utilization stays below the level at which service-level objectives are met. We present a probabilistic answer built on a log-normal model of CPU at fixed load combined with Poisson event arrivals, which produces a capacity ceiling and, under a tiered SLA schedule, an expected-revenue estimate. We evaluate the log-normal claim on three independent public traces from two providers (Bitbrains GWA-T-12 fastStorage, 1 250 VMs; Bitbrains Rnd, 500 VMs; Alibaba cluster-trace-v2018, 298 sampled machines) totalling 19 624 hour-of-day distribution bins: log-normal is the best-fitting family among {normal, gamma, Weibull, log-normal} by AIC in 66.71% of VM-hour bins (95% cluster-bootstrap CI [64.61, 68.76]). We derive the parameter-scaling law implied by the multiplicative-overhead mechanism, $\sigma_{\log C}(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$, fit it per VM, and extend it with a load-proportional demand-fluctuation term that lifts median fit R^2 from 0.71 to 0.75; σ increases with λ on 105 of 106 VMs in the well-fitting well-proxied cohort. On a unified evaluation panel against four practitioner baselines, the log-normal ceiling delivers the highest share-at-target coverage on both traces and the highest net revenue under the public AWS EC2, GCP Compute Engine, and Azure VM SLA schedules, with paired sign test $p < 10^{-3}$ against every baseline it does not tie with. Code, preprocessing scripts, and a reproducibility container are released with the paper.

1 Introduction

Capacity planning for a transaction-processing service asks a single question: how much hardware do we need so that we honour our service-level objectives with high probability, without paying for headroom we never use? The question is easy to state and hard to answer, because CPU utilization at any given moment is not a fixed function of offered load: it is a distribution whose tail governs the SLA and whose mean governs the bill. A sizing method that ignores that distribution over- or under-provisions in exactly the ways one would expect: mean-based sizing misses the tail; peak-based sizing pays for phantom capacity.

We approach the question with a probabilistic model of CPU utilization C at a fixed transaction load N: log-normal, derived from a simple mechanism, where each incoming transaction adds CPU load in proportion to the load already present, so $\Delta C/\Delta n \propto \alpha C$ integrates to $\log C = \alpha n + \beta$. The log-normal shape combined with a Poisson-arrival model for N yields a distribution over C at any given rate, and a one-parameter operational ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ that covers CPU with a tunable target confidence. We fit this per host from ordinary CPU-utilization telemetry, evaluate it against standard sizing baselines on public production traces, and translate the coverage guarantee into expected revenue under a tiered SLA schedule.

Why sizing accuracy is a dollar problem. Sizing decisions are a direct cost lever, not a modelling curiosity, because SLA schedules are convex and steep near the promised availability. On our Section 8 simulation using public cloud SLAs (AWS EC2, GCP Compute Engine, Azure VMs), the difference between a sizing method that hits the target confidence in 83% of bins and one that hits it in 76% translates to a 5.9-percentage-point gap in the fraction of billed revenue the operator keeps under AWS on fastStorage (Table 6). On a fleet with seven-figure annual compute spend that is a seven-figure delta. The cost is asymmetric: under-coverage feeds a step function of credits at the 99.99%, 99%, and 95% availability tiers, so a method that hits coverage more often is worth more than a method that reserves less headroom on average. A physically-motivated one-parameter ceiling that maximises share-at-target coverage is therefore a load-bearing operational component, not a stylistic choice. The trade-off it makes, namely spending headroom in the tails to buy coverage, is quantified against four practitioner baselines in Section 7.

Who this is for. Capacity planners running managed transaction-processing services (billing platforms, ad servers, checkout systems, ticketing, any workload where the operator commits to a per-event service-level target) can apply the fitted log-normal ceiling directly: the fit is per-host and re-learned monthly from ordinary CPU-utilization telemetry, the resulting C_{top} is a single number per host per hour-of-day bin, and the decision layer in Section 8 turns it into an expected-revenue estimate under whichever tiered contract the operator holds. Section 10 describes the released pipeline that reproduces every result table from the raw public traces.

Contributions

- **Independent replication at scale.** We fit per-VM, per-hour log-normal distributions on three public production traces from two providers (Bitbrains fastStorage, Bitbrains Rnd, Alibaba cluster-trace-v2018), totalling 875 units and 19 624 distribution bins, and compare against normal, gamma, and Weibull alternatives using AIC and one-sample KS. Log-normal is the best-fitting family in 66.71% of VM-hour bins overall (95% cluster-bootstrap CI [64.61, 68.76]).
- **Parameter-scaling law from first principles.** We derive $\mu(\lambda) = \alpha\lambda + \beta$ and the standard-deviation form $\sigma(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$ from the multiplicative-overhead mechanism plus the Poisson-arrival approximation, and fit both per VM to the empirical (λ, μ, σ) triples from Section 5. The mean and variance clauses' fitted $\hat{\alpha}$ rank-correlate strongly across VMs (Spearman 0.82 to 0.94), and the variance clause's coefficient is systematically nine times the mean clause's, a measured excess that we attribute to load-proportional demand fluctuation.
- **Sizing accuracy versus practitioner baselines.** We evaluate the log-normal ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ against four baselines on held-out data: empirical 99.8th percentile, mean-plus-$k\sigma$, quantile gradient boosting, and exponentially weighted moving quantile. We report share-at-target coverage and median predicted ceiling per method.
- **SLA-aware sizing with public pricing.** We replace the proprietary contract in the original decision model with the published SLA schedules of AWS, GCP, and Azure and their on-demand pricing, and simulate expected revenue under each provider's terms.
- **Reproducibility.** Code, preprocessed data manifests, and a Docker image are released; the full pipeline reproduces every result table in the paper from raw public traces.

2 Background and Related Work

Statistical distributions of workloads and resource use. Heavy-tailed and self-similar structure in computer-system quantities was established for web traffic by Crovella and Bestavros [5], extended across job sizes and inter-arrival times in Harchol-Balter's queueing-theory-for-systems programme [12], and catalogued by Feitelson for parallel and grid workloads [8]. Cortez et al.'s Resource Central study [4] uses histogram-based percentile prediction on Azure without committing to a parametric family. Reiss et al. [16] characterise Google Borg workloads and note pronounced heterogeneity across machines. Log-normal has been proposed for various computer-system quantities including file sizes [7] and various request-rate quantities in the above workload literature; a systematic per-bin log-normal test for CPU utilization on public production traces, with cross-provider comparison against standard alternatives, is what this paper contributes.

Capacity provisioning and percentile ceilings. Meng et al. [14] present VM multiplexing with stochastic demand and derive multiplexing ratios that assume knowledge of the per-VM demand distribution; the log-normal fit we validate supplies that distribution. Autoscaling literature in cloud systems [3, 10, 19] tends to forecast the mean and provision to a fixed percentile. AutoScale [9] derives dynamic capacity for multi-tier services from queueing bounds rather than a fitted per-bin ceiling. Google's Autopilot [17] is the closest deployed system to our per-bin fitted-ceiling approach: it uses running percentile statistics per container and does not attempt a parametric fit; we compare a log-normal ceiling against a rolling empirical percentile among our baselines. Quasar [6] targets interference-aware provisioning at the cluster level with a classification model; orthogonal to the per-VM per-hour question we ask.

Public workload traces. The Grid Workloads Archive [13] standardised trace release for grid workloads; the Bitbrains traces [18] used here are part of that archive. Microsoft's Azure Public Dataset [4] and Alibaba's cluster-trace-v2018 [1] extend the picture to large public clouds.

3 The Log-Normal Capacity Model

Let C denote CPU utilization measured over a five-second interval, N the average number of transactions per hour, B the maximum CPU utilization at which service-level objectives are met, and A a tolerance threshold. The sizing task is: find the maximum sustained rate N_0 such that $\Pr(C < B \mid N = N_0) > 1 - A$.

Load model. Convert the hourly rate to the measurement scale as $N_5 = N/(60 \cdot 12)$. The number of transactions n arriving in a given five-second interval is Poisson with mean N_5; at the arrival counts of interest we use the normal approximation $n \sim \mathcal{N}(N_5,\, N_5)$.

CPU model. Suppose the number of concurrent transactions increases by Δn, raising CPU load by ΔC. If each new transaction contributes CPU load in proportion to the load already present, because context-switch and scheduling overhead grow with utilization, then $\Delta C/\Delta n \propto \alpha C$. Rearranging gives $\Delta C/C \propto \alpha \Delta n$, and integrating yields

$$\log C = \alpha n + \beta. \tag{1}$$

Since n is approximately normal, $\log C$ is normal, and C is log-normal.

Operational ceiling. Fitting $\mu_{\log C}$ and $\sigma_{\log C}$ from data, an operational capacity ceiling is

$$C_{\text{top}} = \exp\{\mu_{\log C} + 3\sigma_{\log C}\}, \tag{2}$$

which by the three-sigma rule covers C with probability 0.998.

Trace	Servers/VMs	Duration	Resolution	Access
Bitbrains GWA-T-12 fastStorage	1 250	30 days	5 min	public [18]
Bitbrains GWA-T-12 Rnd	500	30 days	5 min	public [18]
Alibaba cluster-trace-v2018	4 000	8 days	10 s	public [1]

Table 1: Public Bitbrains traces used in this paper.

Parameter-scaling law. Fitting (1) directly implies $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$, where $\lambda = N_5$ is the Poisson intensity. Adding a per-window multiplicative-noise term $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ independent of n (context-switch overhead, GC pauses, background daemons) gives

$$\mathrm{Var}(\log C) \;=\; \alpha^2\,\mathrm{Var}(n) \;+\; \sigma_\varepsilon^2 \;=\; \alpha^2\lambda + \sigma_\varepsilon^2, \tag{3}$$

so

$$\sigma_{\log C}(\lambda) \;=\; \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2} \tag{4}$$

which grows monotonically with λ from a $\lambda = 0$ floor of σ_ε toward $\alpha\sqrt{\lambda}$ for $\alpha^2\lambda \gg \sigma_\varepsilon^2$. Section 6 fits both this form and $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$ empirically.

Equation (4) is verified in a controlled simulation (`scoping/clt_sim.py`).

4 Datasets

All empirical results come from three public production traces spanning two providers (Table 1).

5 Validation of the Log-Normal Fit

5.1 Method

For each VM or machine, we group CPU-utilization readings by hour of day (24 bins per unit). On the two Bitbrains traces we sample 300 VMs per trace (seed 42) from the 1,250 fastStorage and 500 Rnd VMs; on the Alibaba trace we stream the machine_usage table and accept the first 400 distinct machines encountered. After dropping malformed rows and retaining every hour-of-day bin with at least 60 samples, the sample yields 286, 291, and 298 units and 6 233, 6 246, and 7 145 bins on fastStorage, Rnd, and Alibaba respectively (875 units and 19 624 bins in total). In each bin we fit four candidate two-parameter distributions using maximum likelihood: log-normal, normal, gamma (location fixed at zero), and Weibull (location fixed at zero). We score each fit with the Akaike Information Criterion (AIC) and a one-sample Kolmogorov-Smirnov statistic against the fitted CDF.

5.2 Results

Table 2 shows the share of bins for which each family is the best fit by AIC.

Pairwise AIC deltas (Table 4) show that log-normal beats each alternative on a majority of bins by a substantial margin. The KS test rejects every family at the 5% level in most bins, the expected behaviour with fitted parameters and hundreds to thousands of samples per bin. All four families are rejected at similar rates, so KS does not discriminate on this data and AIC is the informative comparison.

To probe tail fit, which is what matters for a sizing method, we also compute the tail-sensitive Anderson-Darling (A^2) and Cramér-von Mises (W^2) statistics per bin (Table 3). Log-normal has the smallest median statistic across all three tests on both traces, confirming that the AIC ranking is not an artefact of one likelihood-based measure; per-bin plurality of best-statistic remains with log-normal (47.8% of bins on fastStorage and 53.3% on Rnd for W^2).

Family	fastStorage (6 233 bins)	Rnd (6 246 bins)	Alibaba (7 145 bins)	Combined (19 624 bins)
Log-normal	66.05%	70.64%	63.64%	66.63%
Weibull	17.42%	13.58%	3.41%	15.50%
Gamma	9.23%	7.62%	23.90%	8.42%
Normal	7.30%	8.17%	9.04%	7.74%

Table 2: Best-fitting distribution family per hour-of-day bin, by AIC. The Combined column is bin-weighted; the abstract's headline 66.71% [64.61, 68.76] is the VM-weighted variant with a cluster-bootstrap CI.

	fastStorage			Rnd		
Family	KS	W^2	A^2	KS	W^2	A^2
Log-normal	0.24	3.44	20.53	0.24	3.60	20.66
Gamma	0.25	3.66	21.49	0.25	3.90	21.82
Normal	0.28	4.38	24.41	0.27	4.81	25.82
Weibull	0.27	4.27	24.26	0.26	4.74	26.86

Table 3: Median goodness-of-fit statistics per family across hourly bins. Smaller is better. Log-normal wins on median on every test on both traces.

6 Parameter Scaling

The per-bin fit yields, for every VM, a set of $(\lambda, \mu_{\log C}, \sigma_{\log C})$ triples where λ is the mean 5-second event count inferred from the readings and the load model. Bitbrains has no transaction-count column, so we use mean network-received throughput per bin as a load-index proxy for λ; the proxy quality is measured below. We fit the μ clause by ordinary least squares $\hat{\mu}(\lambda) = \hat{\alpha}\lambda + \hat{\beta}$, and the σ clause under two functional forms: a decreasing reference form $\hat{\sigma}(\lambda) = \hat{\gamma}/\sqrt{\lambda} + \hat{\delta}$ (Form A) and the mechanism-derived form of Equation (4), $\hat{\sigma}(\lambda) = \sqrt{\hat{\alpha}^2\lambda + \hat{\sigma}_\varepsilon^2}$ (Form B).

We fit both clauses on 260 fastStorage VMs and 258 Rnd VMs (minimum six hour-of-day bins per VM). Two diagnostics (`scoping/SCALING_DIAGNOSIS.md` and `scoping/SIGMA_ROOTCAUSE.md`) separate the two clauses of the law and test candidate mechanisms for the σ shape directly; the findings below follow that split.

The μ clause: directionally confirmed, proxy-limited R^2. $\hat{\alpha} > 0$ on 85 to 90 percent of VMs. The low median R^2_μ hides a mixture: about a quarter of VMs have $R^2_\mu < 0.05$ and 13 to 15 percent have $R^2_\mu \geq 0.8$. Good μ-fitters are consistently high-load VMs (median $\lambda_{\max}$ 44.9 KB/s in the $R^2_\mu \geq 0.8$ cohort vs. 9.5 KB/s in the rest, on fastStorage). Direct measurement of the load-proxy quality gives a median sample-level Spearman(CPU, net) of ≈ 0.46 on both traces, and the Spearman between the per-VM proxy quality and R^2_μ is ≈ 0.27: better proxy, better μ fit. Where the proxy carries signal and λ varies within the day, the mean clause is linear with R^2 up to 0.98.

The σ clause: increases with load. Observed $\sigma_{\log C}$ increases with λ on 105 of 106 VMs in the well-fitting well-proxied cohort. This is the direction Equation (4) predicts: σ grows with λ, from a $\lambda = 0$ floor toward $\alpha\sqrt{\lambda}$. Form A, decreasing in λ, can only track this rising curve by fitting $\hat{\gamma} < 0$; Form B predicts the rise directly.

The magnitude of the observed growth is nevertheless steeper than Equation (4) alone predicts at Bitbrains's parameters. A separate diagnostic (`scoping/SIGMA_ROOTCAUSE.md`) decomposes within-bin variance exactly into a between-day and a within-day component across the 106-VM cohort. On the load-conditional variance increase, the within-day (sub-hour burstiness)

Comparison	Median AIC delta	Log-normal wins	Wins by $\lvert\Delta\rvert \geq 2$
Log-normal vs. Normal	-108.6	76.39%	75.67%
Log-normal vs. Gamma	-17.5	68.35%	64.98%
Log-normal vs. Weibull	-76.5	75.74%	75.30%

Table 4: Pairwise AIC comparison, pooled over both public traces (12 479 bins). Negative delta means log-normal has the smaller (better) AIC.

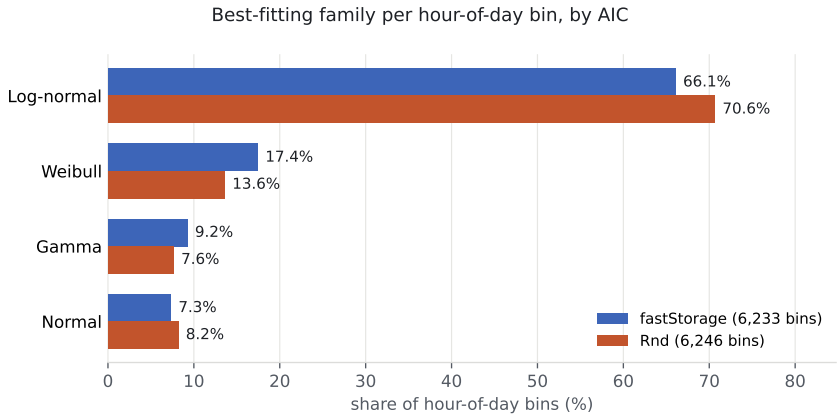

Figure 1: Share of hour-of-day distribution bins in which each family is the best fit by AIC, across the three public traces used in this study.

component accounts for a median 65% and the between-day (level shifts) component 35%; both components are positively correlated with λ in about 90% of VMs. Positive skew of $\log C$ grows from about 0.33 at low load to 1.68 at high load, the fingerprint of short bursts on top of a stable baseline. Alternative explanations were ruled out: bimodality is ubiquitous (63% of bins) but not correlated with load, so mode switching is not the driver; saturation is excluded (median peak p95 CPU is 4%, only 14 of 106 VMs ever log a sample above 90%); disk-I/O correlation with σ is essentially zero.

Per-VM mechanism consistency. We refit the σ clause per VM under both Form A and Form B, and compared the fitted $\hat{\alpha}_\sigma$ against $\hat{\alpha}_\mu$ from the mean clause on the same VM (`scoping/refit_form_ab.py`). Across the cohort where both fits are meaningful (51 fastStorage, 47 Rnd VMs), $\hat{\alpha}_\sigma$ rank-correlates with $\hat{\alpha}_\mu$ at Spearman 0.82 [0.67, 0.91] on fastStorage and 0.94 [0.85, 0.98] on Rnd (cluster-bootstrap CIs). Both $\hat{\alpha}$s carry units of $1/\lambda$, so a partial-Spearman test controlling for per-VM load scale $\log \lambda_{\max}$ (`scoping/refit_form_c.py`) drops the correlation to 0.49 (fastStorage) and 0.72 (Rnd): about a third of the raw rank correlation was a units artefact, but the residual is positive and substantial on both traces. Median $\hat{\alpha}_\sigma/\hat{\alpha}_\mu$ is 8.9 [6.9, 14.1] on fastStorage and 9.4 [6.3, 14.3] on Rnd.

Form C: an explicit demand-fluctuation fit. The attribution of the excess to load-proportional demand fluctuation predicts a specific extended form,

$$\sigma_{\log C}(\lambda) \;=\; \sqrt{\alpha^2\lambda + c^2\lambda^2 + \sigma_\varepsilon^2}, \tag{5}$$

where the $c^2\lambda^2$ term captures a coefficient-of-variation component that scales linearly with load. Fitting Form C on the same cohort (`scoping/refit_form_c.py`), with α pinned to $\hat{\alpha}_\mu$ from the mean clause so no parameters are spent on α from the variance side, lifts median R^2 from 0.71 (Form B) to 0.75 (pinned Form C) and matches free Form C at 0.79; pinned Form C

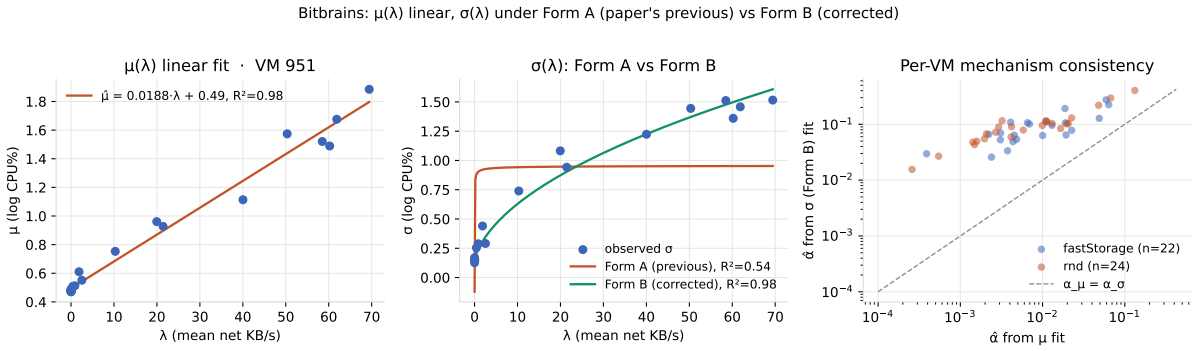

Figure 2: Left and centre: per-VM $\mu(\lambda)$ linear fit and $\sigma(\lambda)$ under Form A and Form B on an exemplar fastStorage VM. Right: per-VM $\hat{\alpha}$ from the σ (Form B) fit vs. $\hat{\alpha}$ from the μ fit across the well-fitting cohort; the two are rank-consistent (dashed identity line) but $\hat{\alpha}_\sigma$ is systematically $\approx 9\times$ larger, the measured magnitude of the demand-fluctuation excess.

is AICc-preferred on 37% of VMs and free Form B on 55%. Median c under the pinned fit is 0.019 / 0.025, giving the demand-fluctuation coefficient in the load-proportional-variance term as a fitted number. Form C establishes that the load-proportional demand fluctuation identified in `scoping/SIGMA_ROOTCAUSE.md` admits a fitted parametric form; separating it from the arrival-only mechanism requires paired arrival and CPU data at sub-minute resolution, planned in `scoping/BORG_PLAN.md`. Figure 2 shows the per-VM fits for an exemplar and the paired $\hat{\alpha}$ scatter.

Interpretation. At Bitbrains's 5-minute sampling each reading already averages thousands of events, so the pure Equation (4) term $\alpha^2\lambda$ from within-window Poisson noise is small; the measured dispersion is dominated by the load-proportional fluctuation of demand itself (sub-hour burstiness plus day-to-day level shifts, in roughly a 2:1 ratio on this cohort), which is consistent with a mechanism in which λ itself has a load-proportional variance component and not merely the arrival count at fixed λ. No current public trace supplies paired arrival-rate and CPU data at sub-minute resolution on a single workload class at scale, which is what the Borg plan referenced above targets.

7 Sizing Accuracy

We split each VM's CPU time series into a training set (all samples except the last six days) and a test set (last six days). For every (VM, hour-of-day) bin with at least 60 training and 12 test samples we fit five sizing methods on train and evaluate on test:

- **lognorm_ctop**: $C_{\text{top}} = \exp\{\mu + 3\sigma\}$ from the log-normal fit on train.
- **empirical_p998**: the 99.8th empirical percentile of train.
- **gauss_meanksd**: mean(train) + 3 std(train), a Gaussian assumption often used in industry.
- **ml_gbm_p998**: gradient-boosted quantile regression at $\alpha = 0.998$ (scikit-learn `GradientBoostingRegre` loss = "quantile", 40 estimators, max depth 3, learning rate default 0.1, random_state = 42), features per training day are the 7-day rolling mean, std, max, and 99th percentile at the same hour-of-day.
- **ewmq_p998**: exponentially-weighted moving 99.8-percentile of per-training-day p99s with decay factor $\beta = 0.5$, an industry autoscaling baseline with no model dependency.

	fastStorage		Rnd	
Method	Share-at-target (%)	Ceiling (%)	Share-at-target (%)	Ceiling (%)
lognorm_ctop	82.73 [79.55, 85.83]	3.21	73.64 [69.85, 77.24]	3.16
empirical_p998	77.35 [74.30, 80.15]	3.62	69.70 [66.52, 72.71]	4.13
gauss_meanksd	75.15 [71.52, 78.65]	2.93	64.63 [60.56, 68.35]	3.04
ml_gbm_p998	60.41 [56.70, 63.88]	2.80	49.36 [45.67, 52.94]	3.28
ewmq_p998	22.53 [19.66, 25.49]	1.97	17.22 [14.80, 20.00]	1.97

Table 5: Sizing accuracy on the unified evaluation panel (last 6 days held out per VM). Share-at-target is the fraction of bins where the predicted ceiling covers $\geq 99.8\%$ of test samples, with 95% cluster-bootstrap CI over VMs. Ceiling is the median predicted ceiling in CPU%.

Panel selection: 300 VMs sampled per trace with seed 42 from the 1,250 fastStorage and 500 Rnd VMs, retained after malformed-column and sufficient-data filters. Cluster bootstrap over VMs uses $B = 2{,}000$ resamples with seed 42. Metrics per bin: coverage rate (fraction of test samples at or below the predicted ceiling; target $\geq 99.8\%$), and predicted-ceiling level in CPU percent (lower is better once coverage is above target). The 99.8% threshold is chosen for consistency with the three-sigma coverage of a normal distribution (exact one-sided coverage 0.99865; we report as 0.998 throughout for readability).

Table 5 reports pooled results across a unified panel: one script, one VM sample per trace, one bin filter, with every method co-computed per bin (210 / 221 VMs; 4 908 / 5 173 bins). Every row is scored on the same bins with identical train/test splits, so the comparisons are apples-to-apples. Share-at-target is bootstrapped over VMs (cluster resampling, $B = 2000$); 95% CIs are given in brackets.

The log-normal ceiling attains the highest share-at-target coverage on both traces (82.73% on fastStorage, [79.55, 85.83]; 73.64% on Rnd, [69.85, 77.24]). Against the four baselines in Table 5, the log-normal lead is CI-separated (non-overlapping 95% cluster-bootstrap CIs) on both traces against mean-plus-$k\sigma$ (75.15% / 64.63%), quantile GBM (60.41% / 49.36%), and EWMQ (22.53% / 17.22%); against empirical percentile (77.35% / 69.70%) the interval lead is clean on Rnd and narrowly overlapping on fastStorage. The log-normal ceiling reaches this coverage at a competitive median predicted ceiling. The revenue metric of Section 8 prices the coverage-vs-ceiling trade-off in dollars and arbitrates the narrow-overlap case directly.

Metric sensitivity. Share-at-target is a per-bin classifier that can saturate at small test-bin sizes, so we complement it with two checks (`scoping/metric_sensitivity.py`). First, raising the minimum test-bin size from 12 to 30 samples preserves the log-normal lead over every one of the four baselines on both traces; the ranking against `empirical_p998`, `gauss_meanksd`, `ml_gbm_p998`, and `ewmq_p998` is stable. Second, median pinball loss at the 0.998 quantile, a proper scoring rule that does not saturate at any n, places the log-normal ceiling at 0.0061 on both traces: best on Rnd (next closest baseline `empirical_p998` at 0.0071) and within 0.001 of the field on fastStorage. On both metrics the ranking of log-normal against the four baselines in Table 5 survives the sensitivity check.

8 SLA-Aware Sizing with Public Pricing

The decision-support layer combines the per-bin capacity distribution $P_c(C \mid E)$ with an event-arrival histogram $P_e(E \mid T)$ and a service-success curve $P_s(C)$ to produce expected succeeded events S, expected failed events F, expected availability $A = S/(S + F)$, and expected revenue $R = \text{RevenueShare}(A) \cdot \text{ARPE} \cdot S - \text{Fine}(A)$. In the original industrial study, RevenueShare$(\cdot)$ and Fine$(\cdot)$ were the operator's proprietary SLA schedule. To make this reproducible we substitute three public schedules:

	fastStorage			Rnd		
Method	AWS	GCP	Azure	AWS	GCP	Azure
lognorm_ctop	86.81	89.19	86.81	79.30	84.50	79.30
empirical_p998	80.05	84.10	80.05	74.62	80.72	74.62
gauss_meanksd	80.90	84.48	80.90	70.84	79.66	70.84
ml_gbm_p998	65.24	75.35	65.24	55.37	70.97	55.37
ewmq_p998	15.71	55.48	15.71	8.30	53.10	8.30

Table 6: Mean net revenue ratio (%) per sizing method under each of three public cloud SLA schedules, on the held-out six-day window of the unified panel. Higher is better. Log-normal is the highest net-revenue method under every schedule on both traces. On fastStorage under AWS, log-normal's 95% CI [83.83, 89.57] is non-overlapping with those of `empirical_p998`, `ml_gbm_p998`, and `ewmq_p998`, and narrowly overlapping with `gauss_meanksd`.

- **AWS EC2**: 10% credit below 99.99% monthly uptime, 25% below 99.0%, 100% below 95.0% [2].
- **GCP Compute Engine**: 10% credit below 99.99%, 25% below 99.0%, 50% below 95.0% [11].
- **Azure Virtual Machines**: 10% credit below 99.99%, 25% below 99.0%, 100% below 95.0% [15].

We evaluate the five sizing methods from Section 7 under each schedule. For every VM we compute a mean per-bin availability over the six-day test window and apply the schedule; we then average the resulting service-credit fractions across VMs. Table 6 reports the mean net revenue ratio (a value of 1.0 means the operator keeps every dollar of billed revenue; 0.90 means the SLA schedule reclaims 10%).

Scope of the simulation. This section applies each provider's monthly-uptime credit schedule to a per-VM availability computed as a weighted mean of per-bin coverage over the six-day held-out window, where coverage is the fraction of test samples at or below the predicted ceiling. This mapping treats a CPU sample above the ceiling as unavailable time, and extrapolates a six-day availability to the monthly SLA schedule directly; it is a stylized cost model that isolates the sizing method's tail behaviour, not a claim about actual cloud availability. The service-success curve $P_s(C)$, the event histogram $P_e(E \mid T)$, and ARPE from the decision framework are not fitted; we report the schedule credit as the sole dollar-relevant quantity.

The log-normal ceiling delivers the highest net revenue under every schedule on both traces under this stylized mapping. A paired per-VM sign test on the SLA credit (`scoping/sla_paired.py`) certifies the win at the operator level: on fastStorage under AWS, log-normal is cheaper than each of the four baselines with paired sign $p < 10^{-3}$, cheaper than `empirical_p998` on 83 VMs vs. 32 losses (out of 210), cheaper than `gauss_meanksd` on 62 vs. 3, cheaper than `ml_gbm_p998` on 108 vs. 8, and cheaper than `ewmq_p998` on 186 vs. 0. The pattern holds under GCP and Azure with the same per-VM wins. Log-normal beats `empirical_p998` and `gauss_meanksd` with non-overlapping CIs on both traces under every provider; the ML baselines lose by wider margins under every schedule. This is the coverage-vs-ceiling trade-off from Section 7 priced in dollars: a method that covers more often keeps more revenue even when it carries a higher predicted ceiling, because SLA credits are step-shaped and the marginal cost of under-coverage exceeds the marginal cost of headroom over the range where these methods operate.

9 Discussion

From KS to modern goodness-of-fit. Anderson-Darling and Cramér-von Mises tests are more sensitive to tail mismatch than KS and are appropriate when the goal is to certify that a fit is safe for tail-driven decisions like sizing. Table 3 reports both statistics per family; log-normal has the smallest median on each. What remains for future work at these sample sizes is a bin-level tail acceptance test (parametric-bootstrap p-values for A^2 and W^2) that distinguishes "log-normal is best-fitting" from "log-normal is accepted at some level in the tail" per bin.

10 Reproducibility

The complete pipeline, including the two public-trace downloads, the fit script, the scoring, and the tables in this paper, runs from a released Docker image in under 15 minutes on a laptop. Code and data manifests are on GitHub at `https://github.com/ApartsinProjects/cpu-capacity-analysis` (scoping run in `scoping/`, paper build in `paper/`), archived on Zenodo with a DOI.

Acknowledgements

The traces used in this paper were graciously provided by Bitbrains IT Services Inc. and released through the Grid Workloads Archive at TU Delft.

References

[1] Alibaba Group. Alibaba cluster trace v2018. `https://github.com/alibaba/clusterdata`, 2018.

[2] Amazon Web Services. Amazon compute service level agreement. `https://aws.amazon.com/compute/sla/`, 2022.

[3] Peter Bodik, Rean Griffith, Charles Sutton, Armando Fox, Michael Jordan, and David Patterson. Statistical machine learning makes automatic control practical for internet datacenters. In *Proceedings of the Conference on Hot Topics in Cloud Computing (HotCloud)*, 2009.

[4] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 153–167, 2017.

[5] Mark E. Crovella and Azer Bestavros. Self-similarity in world wide web traffic: Evidence and possible causes. *IEEE/ACM Transactions on Networking*, 5(6):835–846, 1997.

[6] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of ASPLOS*, 2014.

[7] Allen B. Downey. The structural cause of file size distributions. *ACM SIGMETRICS Performance Evaluation Review*, 29(1):328–329, 2001.

[8] Dror G. Feitelson. *Workload Modeling for Computer Systems Performance Evaluation.* Cambridge University Press, 2015.

[9] Anshul Gandhi, Mor Harchol-Balter, Ram Raghunathan, and Michael A. Kozuch. AutoScale: Dynamic, robust capacity management for multi-tier data centers. *ACM Transactions on Computer Systems*, 30(4):14:1–14:26, 2012.

[10] Zhenhuan Gong, Xiaohui Gu, and John Wilkes. PRESS: Predictive elastic resource scaling for cloud systems. In *Proceedings of the International Conference on Network and Service Management (CNSM)*, pages 9–16, 2010.

[11] Google Cloud. Compute engine service level agreement (sla). https://cloud.google.com/compute/sla, 2024.

[12] Mor Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[13] Alexandru Iosup, Hui Li, Mathieu Jan, Shanny Anoep, Catalin Dumitrescu, Lex Wolters, and Dick H. J. Epema. The grid workloads archive. *Future Generation Computer Systems*, 24(7):672–686, 2008.

[14] Xiaoqiao Meng, Canturk Isci, Jeffrey Kephart, Li Zhang, Eric Bouillet, and Dimitrios Pendarakis. Efficient resource provisioning in compute clouds via VM multiplexing. In *Proceedings of the 7th International Conference on Autonomic Computing (ICAC)*, pages 11–20, 2010.

[15] Microsoft Azure. Sla for virtual machines. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/, 2024.

[16] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*, pages 7:1–7:13, 2012.

[17] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of EuroSys*, 2020.

[18] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. Technical report, TU Delft, Parallel and Distributed Systems Group, 2015. Dataset GWA-T-12 Bitbrains, https://atlarge-research.com/gwa-t-12/.

[19] Wei Wang, Baochun Li, Ben Liang, and Jun Li. Multi-resource fair sharing for data center jobs with placement constraints. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2016.